

模拟卷 07-2025 风格题目与详解

目录

1	编译原理 2025 风格模拟卷 07（题目与详解）	2
2	A 卷题目	2
3	B 详解	4
4	C 复盘清单	9

1 编译原理 2025 风格模拟卷 07 (题目与详解)

1.1 卷面说明

本卷按 Slides/Slides/handout 与三份 Recap PPT 更新, 主线覆盖前端自动机与 LL、中端 IR/SSA、数据流、符号执行、后端目标代码、寄存器分配、指令调度以及过程间/指针分析。总分 100 分。

2 A 卷题目

2.1 一、词法分析 (15 分)

正则表达式:

$$(x|y)^* x (x|y)$$

1. 使用 Thompson 构造法构造等价 NFA。
2. 使用子集构造法构造 DFA, 写明 DFA 状态含义。
3. 最小化 DFA。

2.2 二、LL 语法分析 (15 分)

文法:

```
H -> I J
I -> I arrow | var
J -> eq | colon
```

1. 消除左递归。
2. 写 FIRST 和 FOLLOW。
3. 写预测分析表。
4. 判断消除左递归后的文法是否为 LL(1)。

2.3 三、指令调度 (15 分)

给定指令:

```

I1: R1 = R2 + R3
I2: R4 = R1 + R5
I3: R2 = R6 + R7
I4: R1 = R8 + R9
I5: R10 = R4 + R1

```

1. 写出指令调度的目的。
2. 写出 RAW、WAR、WAW 的定义。
3. 构建依赖图，WAR/WAW 用虚线表示。
4. 在允许寄存器重命名的条件下给出一个合法重排序列。

2.4 四、支配问题（15 分）

CFG：双分支汇合后出口

```

Entry->A
A->B
A->C
B->D
C->D
D->E
D->F
E->Exit
F->Exit

```

1. 写出 Dom、PostDom、Dom Frontier 的定义。
2. 计算每个节点的 Dom 集和 idom。
3. 写出支配边界。
4. 说明 SSA 中 phi 函数应放在什么位置。

2.5 五、符号执行与 SAT（20 分）

公式：

$$F = ((a7 \text{ and } b7) \text{ or } c7)$$

1. 写出 Path-sensitive、Flow-sensitive、Context-sensitive 的定义。
2. 写出 CNF 的定义。
3. 使用 Tseitin 算法把 F 转成 CNF。
4. 证明 2025 卷中的两组公式同时可满足。

2.6 六、Andersen 指针分析（20 分）

程序：

```
p = &A
q = &B
r = &C
*p = q
q = r
t = *p
```

1. 对每条语句写 Andersen 约束。
2. 根据约束构建闭包。
3. 写出最终 points-to 集合。

2.7 七、PPT 新增重点：SSA 转出（10 分）

SSA 片段：

```
B2 -> B4
B3 -> B4

B2: x1 = 1
B3: x2 = 2
B4: x3 = phi(x1 from B2, x2 from B3)
    y1 = x3 + 1
```

1. 说明 phi 转出时复制语句应插在哪里。
2. 写出本例转出 SSA 后在 B2、B3、B4 中的关键语句。
3. 如果 B2->B4 是 critical edge，应如何处理。

3 B 详解

3.1 一、词法分析详解

正则表达式：(x|y)* x (x|y)

Thompson NFA：起点 q0，终点 q11。按下列转移表画图，q11 画双圈。

```
q4 --x--> q5
q6 --y--> q7
q2 --epsilon--> q4
q2 --epsilon--> q6
```

```

q5 --epsilon--> q3
q7 --epsilon--> q3
q0 --epsilon--> q2
q0 --epsilon--> q1
q3 --epsilon--> q2
q3 --epsilon--> q1
q8 --x--> q9
q1 --epsilon--> q8
q12 --x--> q13
q14 --y--> q15
q10 --epsilon--> q12
q10 --epsilon--> q14
q13 --epsilon--> q11
q15 --epsilon--> q11
q9 --epsilon--> q10

```

子集构造 DFA:

D0:

```

NFA集合 = {q0,q1,q2,q4,q6,q8}
on x -> D1
on y -> D2
终态 = 否

```

D1:

```

NFA集合 = {q1,q2,q3,q4,q5,q6,q8,q9,q10,q12,q14}
on x -> D3
on y -> D4
终态 = 否

```

D2:

```

NFA集合 = {q1,q2,q3,q4,q6,q7,q8}
on x -> D1
on y -> D2
终态 = 否

```

D3:

```

NFA集合 = {q1,q2,q3,q4,q5,q6,q8,q9,q10,q11,q12,q13,q14}
on x -> D3
on y -> D4
终态 = 是

```

D4:

```

NFA集合 = {q1,q2,q3,q4,q6,q7,q8,q11,q15}
on x -> D1
on y -> D2
终态 = 是

```

最小化:

初分组:

终态组 = {D3,D4}

非终态组 = {D0,D1,D2}

最终分组:

G1 = {D3}

G2 = {D4}

G3 = {D0,D2}

G4 = {D1}

每个最终分组对应一个最小 DFA 状态。

3.2 二、LL 语法分析详解

原文法:

```
H -> I J
I -> I arrow | var
J -> eq | colon
```

消除左递归:

```
H -> I J
I -> var I'
I' -> arrow I' | epsilon
J -> eq | colon
```

FIRST:

```
FIRST(H)={ var }
FIRST(I)={ var }
FIRST(I')={ arrow, epsilon }
FIRST(J)={ eq, colon }
```

FOLLOW:

```
FOLLOW(H)={ $ }
FOLLOW(I)={ eq, colon }
FOLLOW(I')={ eq, colon }
FOLLOW(J)={ $ }
```

预测表:

```
M[H,var] = H -> I J
M[I,var] = I -> var I'
M[I',arrow] = I' -> arrow I'
M[I',eq] = I' -> epsilon
M[I',colon] = I' -> epsilon
M[J,eq] = J -> eq
M[J,colon] = J -> colon
```

结论: 表中无多重入口, 消除左递归后的文法是 LL(1)。

3.3 三、指令调度详解

定义:

指令调度在不改变程序语义的前提下重排指令, 减少流水线停顿, 提高并行执行效率。

RAW: 先写后读, 必须保持顺序。

WAR: 先读后写, 写者不能提前。

WAW: 先写后写, 顺序影响最终值。

指令:

```
I1: R1 = R2 + R3
```

```

I2: R4 = R1 + R5
I3: R2 = R6 + R7
I4: R1 = R8 + R9
I5: R10 = R4 + R1

```

依赖:

```

I1 -> I2 RAW on R1
I1 -> I3 WAR on R2
I1 -> I4 WAW on R1
I2 -> I4 WAR on R1
I2 -> I5 RAW on R4
I4 -> I5 RAW on R1

```

寄存器重命名:

```

I3: R2_new = R6 + R7
I4: R1_new = R8 + R9
I5 使用 R1_new

```

重命名后保留 RAW, WAR/WAW 被消除。一个合法调度序列:

```

I1, I3, I4, I2, I5

```

3.4 四、支配问题详解

定义:

Dom(n): 入口到 n 的每条路径都经过的节点集合。

PostDom(n): n 到出口的每条路径都经过的节点集合。

Dom Frontier(n): n 支配某个前驱, 但不严格支配该节点的位置。

CFG 边:

Entry->A

A->B

A->C

B->D

C->D

D->E

D->F

E->Exit

F->Exit

Dom 集:

Dom(Entry)={Entry}

Dom(A)={Entry, A}

Dom(B)={Entry, A, B}

Dom(C)={Entry, A, C}

Dom(D)={Entry, A, D}

Dom(E)={Entry, A, D, E}

Dom(F)={Entry, A, D, F}

Dom(Exit)={Entry, A, D, Exit}

直接支配者:

```
idom(A)=Entry
idom(B)=A
idom(C)=A
idom(D)=A
idom(E)=D
idom(F)=D
idom(Exit)=D
```

支配边界:

```
DF(B)={D}
DF(C)={D}
DF(E)={Exit}
DF(F)={Exit}
```

其余节点 DF 为空集

SSA phi 放置: 若变量在多个基本块中被定义, 则在这些定义块的迭代支配边界处插入该变量的 phi。直观上, phi 放在多

3.5 五、符号执行与 SAT 详解

公式: $F = ((a7 \text{ and } b7) \text{ or } c7)$

引入: $x71 \leftrightarrow (a7 \text{ and } b7)$, $x72 \leftrightarrow (x71 \text{ or } c7)$, 根变量 $x72$ 为真。

CNF:

```
(not x71 or a7)
(not x71 or b7)
(x71 or not a7 or not b7)
(x72 or not x71)
(x72 or not c7)
(not x72 or x71 or c7)
(x72)
```

敏感性定义:

Path-sensitive 区分不同路径。

Flow-sensitive 区分程序点和语句顺序。

Context-sensitive 区分函数调用上下文。

同时可满足证明套用:

$F1 = (a0 \text{ or } \dots \text{ or } a_n \text{ or } c) \text{ and } (b0 \text{ or } \dots \text{ or } b_m \text{ or } \text{not } c)$, $F2 = (a0 \text{ or } \dots \text{ or } a_n \text{ or } b0 \text{ or } \dots \text{ or } b_m)$ 。

$F1$ 为真时, $c=false$ 推出某个 $a_i=true$, $c=true$ 推出某个 $b_j=true$, 所以 $F2$ 为真。

$F2$ 为真时, 若某个 $a_i=true$ 令 $c=false$; 若某个 $b_j=true$ 令 $c=true$, 所以 $F1$ 为真。

3.6 六、Andersen 指针分析详解

程序:

```
p = &A
q = &B
r = &C
*p = q
q = r
```


$t = *p$

约束:

$A \in \text{pts}(p)$

$B \in \text{pts}(q)$

$C \in \text{pts}(r)$

for $v \in \text{pts}(p)$, $\text{pts}(q) \subseteq \text{pts}(v)$

$\text{pts}(r) \subseteq \text{pts}(q)$

for $v \in \text{pts}(p)$, $\text{pts}(v) \subseteq \text{pts}(t)$

闭包:

$\text{pts}(p) = \{A\}$

$\text{pts}(q) = \{B, C\}$

$\text{pts}(r) = \{C\}$

由 $*p=q$ 得 $\text{pts}(A) = \{B, C\}$

由 $t=*p$ 得 $\text{pts}(t) = \{B, C\}$

3.7 七、PPT 新增重点详解

ϕ 转出原则:

把 ϕ 的每个输入改成对应前驱边上的复制。

B2:

$x1 = 1$

$x3 = x1$

B3:

$x2 = 2$

$x3 = x2$

B4:

$y1 = x3 + 1$

如果 B2→B4 是 critical edge, 不能直接把复制插到 B2 末尾。

应先拆边, 插入新的基本块, 在新块中放 $x3=x1$ 。

4 C 复盘清单

词法: Thompson、epsilon-closure、move、终态集合、最小化分组。

LL: 左递归、FIRST、FOLLOW、预测表、多重入口。

IR/SSA: 三地址码、基本块、支配边界、 ϕ 插入与转出。

数据流: 方向、meet、transfer、gen/kill、worklist、IN/OUT。

后端: 目标代码、地址模式、冲突图、spill、指令调度。

符号执行: 敏感性、CNF、Tseitin 子句、同时可满足证明。

过程间/指针: 调用上下文、上下文敏感、Andersen 约束与闭包。